Parking Stay-Duration Classification

San Diego Parking Meter Data: A Binary Classification Study

Abhijith Mohanan

MSc Data Science & Intelligent Analytics

FH Kufstein Tirol, University of Applied Sciences

July 14, 2026

Datasets: City of San Diego Open Data Portal

https://data.sandiego.gov

Abstract

Anyone who has driven around a block three times looking for a parking spot knows how much time gets wasted on something that should be simple. That everyday frustration is the starting point for this project. Using historical parking meter data published by the City of San Diego, a model was built to predict whether a given parking transaction results in a long stay, over 60 minutes, or a short stay, 60 minutes or less. Two public datasets were merged for this: a year of meter transactions from 2020 (just under 2 million records) and a location file describing where each of the city's 4,060 active meters sits. After cleaning both datasets and joining them, 1,474,238 usable transactions and 19 engineered features remained. Four different classifiers were then trained on this data, Random Forest, XGBoost, CatBoost, and a small neural network (MLP), and their performance was compared. All four ended up in the same neighbourhood, above 86% accuracy, which was reassuring rather than surprising given how different these algorithms are under the hood. Because one feature, the transaction amount, turned out to dominate every model's decision-making, all four models were also retrained with that feature removed, to see how much of the accuracy was really coming from it.

Contents

1 Introduction

Parking is one of those problems that sounds trivial until you are stuck in it. In a busy downtown area, a driver circling for an open spot is not just inconvenienced, they are adding to congestion and burning fuel that a slightly smarter system could have saved. Cities have started responding to this with "smart" parking infrastructure: meters and sensors that log every transaction electronically instead of relying on a coin box and a paper log.

San Diego is a good example of this. The city runs a large network of electronic parking meters spread across downtown, uptown, and several surrounding districts, and every payment made at one of these meters is recorded with a timestamp, a location, and a payment method. That transactional record is what this project works with.

It is worth being upfront about what this model can and cannot do. Predicting whether a specific spot is free right now would need live occupancy sensors, and this dataset does not have those. What it does have is a detailed history of when people pay and for how long, and that turns out to be a reasonable stand-in for the question people actually care about: how quickly does parking turn over in a given area at a given time? A zone where most transactions are long stays will free up spots slowly. A zone dominated by short stays turns over fast. So rather than predicting an exact duration in minutes, this project treats the problem as a binary classification task: will this transaction be a **long stay** (over 60 minutes) or a **short stay** (60 minutes or less)?

Objectives

This project had four main goals. The first was building a clean, reliable dataset out of two separate, somewhat messy real-world files. The second was training and comparing several different model types on the same task, rather than settling for a single algorithm. The third was digging into which features actually drive the predictions, not just reporting an accuracy number. The fourth, which followed almost naturally once the feature importance results came in, was checking how the models would perform if the single most dominant feature, the amount a driver paid, were taken away entirely.

Classification was chosen over regression because, in practice, telling a driver or a city planner "this is likely a long stay" is more useful than an exact number of minutes that would carry a lot of uncertainty anyway.

2 Related Work

Most existing work on parking prediction falls into one of two camps. The first relies on dedicated occupancy sensors installed in individual spots, which gives very accurate, real-time results but is expensive to roll out at city scale [1]. The second works from transaction or payment data that cities already collect, which is far cheaper to obtain and easier to scale, even if it is a step removed from true occupancy [2]. This project sits firmly in the second camp.

On the modeling side, Ji et al. [3] found across several cities that simple temporal features, hour of day and day of week in particular, tend to be some of the strongest predictors available, which matches the pattern found in the San Diego data as well. Tree-based ensembles such as Random Forest and gradient boosting have also repeatedly shown strong results on this kind of tabular, structured data [4].

Studies using the well-known UCI Birmingham parking dataset [5] found that city-centre parking behaves quite differently from suburban parking in terms of occupancy patterns. That same contrast shows up clearly in the San Diego data: the City and Downtown zones consistently show far more long-stay behaviour than Uptown or Mid-City, which lines up with what would be expected from a busy commercial core versus a more residential fringe.

3 Dataset Description

3.1 Data Sources

Both datasets used here are published by the City of San Diego on its open data portal (`https://data.sandiego.gov`), freely available under an open government license.

3.2 Parking Meter Transactions (2020)

The first dataset logs every parking payment made in the city during 2020, running from late February through the end of December. As published, it contains 1,953,052 rows across 6 columns, summarised in Table 1.

Table 1: Parking Meter Transactions Dataset Columns

Column	Type	Description
pole_id	String	Unique meter identifier
meter_type	String	Type of meter (all SS in 2020)
date_trans_start	Datetime	Transaction start timestamp
date_meter_expire	Datetime	Meter expiry timestamp
trans_amt	Integer	Amount paid (in cents)
pay_method	String	CASH, CREDIT CARD, or SMART CARD

Cash was still the most common way to pay in 2020 (1,058,818 transactions), with credit card close behind (892,069), and only a small number of smart card payments (2,165). Across the whole year, 4,592 distinct meters logged at least one transaction.

3.3 Parking Meter Locations (Active)

The second dataset is a static snapshot of every active meter in the city: where it is, what zone it belongs to, and a few operating details. It has 4,060 rows and 16 columns; the fields relevant to this project are listed in Table 2.

Table 2: Parking Meter Locations Dataset, Key Columns

Column	Type	Description
pole	String	Meter ID (join key)
zone	String	Parking zone
area	String	Sub-area within zone
latitude	Float	GPS latitude
longitude	Float	GPS longitude
time_limit	String	Maximum parking duration
days_in_operation	String	Operating days
price	String	Hourly rate

Most of these meters sit in Downtown (2,404) and Uptown (1,090), with smaller numbers in Mid-City (348), the City zone (90), Balboa Park (60), Pacific Beach (29), and San Ysidro (25). A handful were tagged Test Zone (13) or Spares (1), which turned out to be internal placeholders rather than real meters. Altogether, meters are spread across 42 distinct sub-areas.

4 Methodology

4.1 Data Preprocessing

Neither dataset was clean straight out of the box, so a fair amount of this project involved simply getting the data into a usable state.

Twelve duplicate transaction rows were found and removed early on. A check for logically invalid rows, where the meter expiry time came before the start time, turned up none. A larger issue was 44,806 rows (about 2.29% of all transactions) where the start and expiry timestamps were identical. Almost all of these were cash payments, and since a genuine zero-minute parking transaction does not really make sense, these were treated as recording errors and dropped rather than having a duration imputed.

On the location side, the "Test Zone" and "Spares" entries mentioned above were removed, along with one row that had GPS coordinates of exactly (0, 0), an obvious placeholder rather than a real location. For the remaining rows, a few fields (`time_limit`, `days_in_operation`, `price`) had missing values here and there. Rather than dropping those rows outright and losing otherwise good data, the gaps were filled with the string "Unknown".

Once both files were cleaned, they were joined on the meter ID (`pole_id` on the transaction side, `pole` on the location side). 3,334 meters appeared in both datasets, and the resulting merged table, which everything from here on is based on, contains 1,474,238 records.

4.2 Feature Engineering

From this merged data, 19 features were engineered, grouped into five rough categories in Table 3.

Table 3: Engineered Features by Category

Category	**Features**
Time-based	hour, day_of_week, month, week_of_year
Business flags	is_weekend, is_morning_peak (8 to 10am), is_midday (11am to 2pm), is_afternoon (3 to 5pm)
Cyclical encoding	hour_sin, hour_cos, dow_sin, dow_cos
Transaction	trans_amt_dollars
Location	price_per_hr, time_limit_mins, operates_weekend, zone_enc, area_enc, pay_method_enc

For the hour and day-of-week features, sine and cosine transforms were used rather than raw integers. Without this, a model would treat 11pm and midnight as far apart on a

number line, when in reality they are one hour apart, so the cyclical encoding corrects for that. One feature deliberately left out was the raw transaction duration itself: since that is exactly what defines the target label, including it would have been straightforward data leakage.

4.3 Target Variable

The target variable, `is_long_stay`, is defined simply as:

$$\texttt{is\_long\_stay} = \begin{cases} 1 & \text{if duration} > 60 \text{ minutes (long stay)} \\ 0 & \text{if duration} \leq 60 \text{ minutes (short stay)} \end{cases} \tag{1}$$

In the cleaned dataset, long stays make up 786,859 records (53.4%) and short stays make up 687,379 (46.6%). That is close enough to a 50/50 split that it was not treated as a serious imbalance problem, though class weighting was still applied in the tree-based models as a precaution.

4.4 Model Selection

The data was split 80/20 into training and test sets using stratified sampling, giving 1,179,390 training rows and 294,848 test rows.

For Random Forest, 100 trees were used with `max_depth=12` and `class_weight='balanced'`. XGBoost was set up with 100 estimators, `max_depth=6`, a `learning_rate` of 0.1, and `scale_pos_weight` set to the ratio of negative to positive samples in the training set. CatBoost used a comparable configuration, 100 iterations, `depth=6`, the same `learning_rate`, and the same `scale_pos_weight` as XGBoost, so that the two gradient boosting models would be on equal footing.

The fourth model was a small MLP with two hidden layers (64 and 32 neurons), ReLU activations, and early stopping. Neural networks are sensitive to the scale of their inputs in a way tree models are not, so all features were standardised with `StandardScaler` before training this one. It is also worth noting that scikit-learn's `MLPClassifier` has no built-in `class_weight` option, so this model handles the (mild) class imbalance less directly than the others do.

5 Experimental Results

5.1 Exploratory Data Analysis

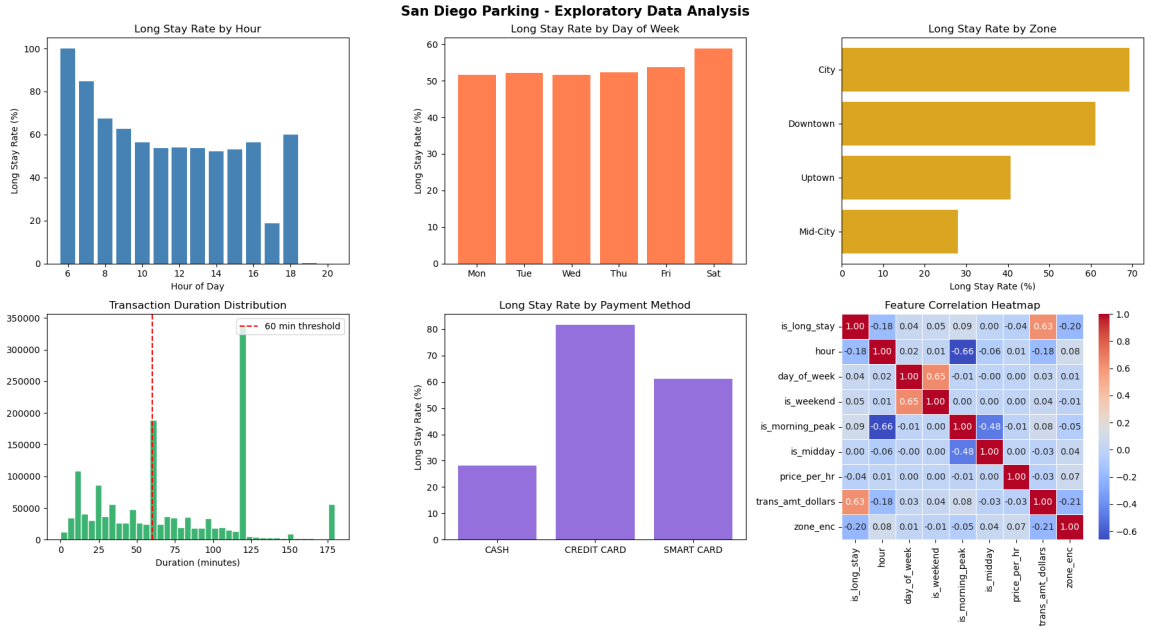

Figure 1: EDA charts: long stay rate by hour, day of week, zone, payment method, duration distribution, and correlation heatmap.

A few patterns stand out once the cleaned data is plotted. Looking at hour of day, the long stay rate peaks at hour 6, close to 100%, so almost every transaction that early in the morning turns into a long stay. Through the middle of the day, roughly 10am to 4pm, the rate settles into a steadier 50 to 55% range. It then drops off sharply around 5pm, down to about 18%, before climbing back up to roughly 60% by 6pm. A likely explanation is that 5pm is dominated by drivers grabbing a quick spot right before enforcement ends for the day.

Day of week tells a quieter story. Weekdays sit around 50 to 55% fairly consistently, and Saturday nudges a bit higher, around 58%. Sunday barely shows up in the data at all, since most meters in San Diego are not even enforced on Sundays.

Zone matters more than might be expected at first glance. The City zone has the highest long-stay rate at around 70%, followed by Downtown at around 62%. Uptown drops to around 40%, and Mid-City is lowest at around 25%, which fits with Mid-City being a more residential, less commercial area.

The overall duration distribution is bimodal rather than smooth: a large cluster of short transactions under an hour, and a second, sharp spike right at 120 minutes, the most common posted time limit. That spike suggests many drivers simply pay for the maximum time allowed up front, rather than estimating how long they will actually need.

Payment method also correlates strongly with stay length. Credit card transactions have the highest long-stay rate, around 80%, well ahead of smart card (around 60%) and cash (around 28%). This is one of the more intuitive patterns in the dataset: paying by card is simply easier for a larger, longer transaction than counting out coins.

Finally, the correlation heatmap confirms what the modeling results later make very clear: `trans_amt_dollars` correlates with `is_long_stay` at 0.63, easily the strongest relationship in the dataset. `hour` shows a much weaker negative correlation of -0.18, meaning later hours lean slightly toward shorter stays, but nowhere near as strongly.

5.2 Model Performance

Table 4 shows how the four models performed on the 294,848 held-out test records.

Table 4: Model Performance Comparison (All Features)

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**
Random Forest	0.8612	0.9387	0.7917	0.8590
XGBoost	0.8640	0.9579	0.7796	0.8596
CatBoost	0.8625	0.9500	0.7800	0.8582
MLP	0.8644	0.9400	0.7900	0.8621

What stands out here is how close these four numbers are. All four models cluster around 86% accuracy and an F1 score near 0.86, despite being fairly different approaches, two gradient boosting variants, a bagged tree ensemble, and a neural network. When models this different agree this closely, it usually means the ceiling is being set by the data itself rather than by any particular algorithm's strengths or weaknesses. The MLP holding its own against XGBoost and CatBoost was not necessarily the expected outcome, yet it clearly does.

5.3 Confusion Matrices

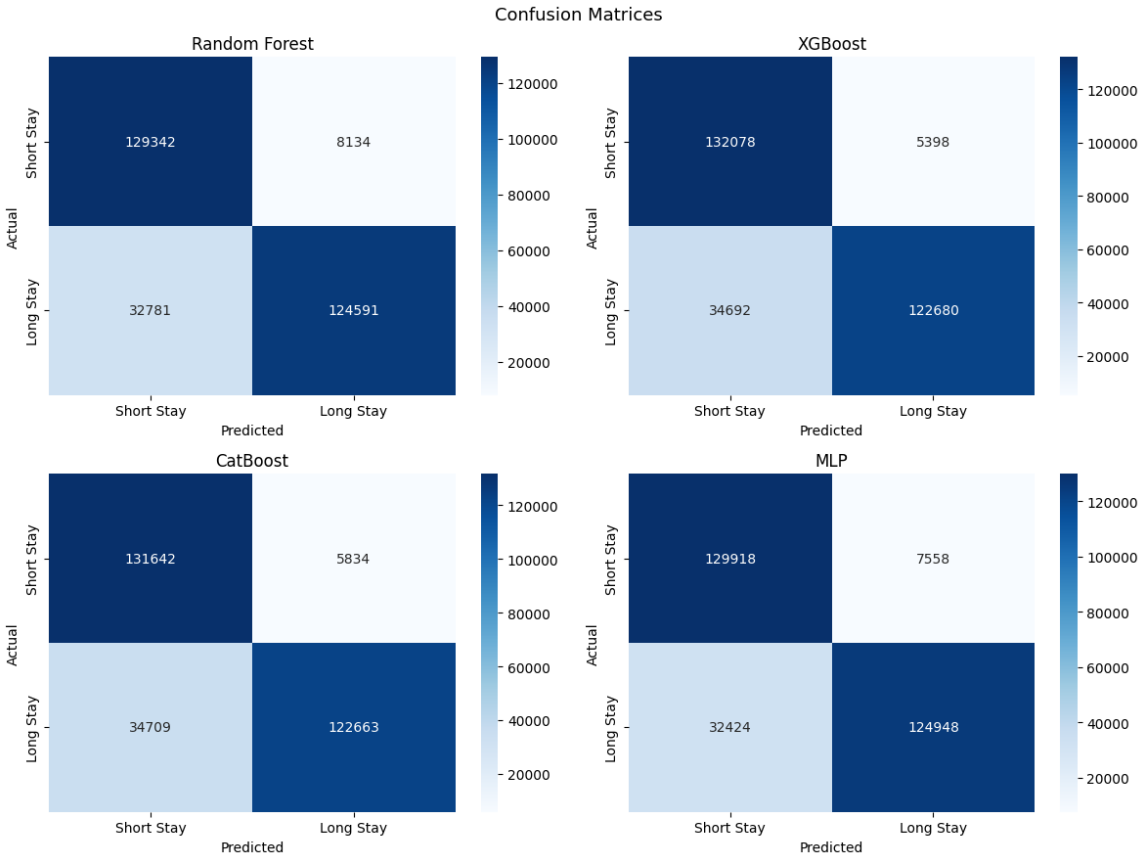

Figure 2: Confusion matrices for all four models (2×2 grid).

Table 5: Confusion Matrix Values, All Four Models (Test Set: 294,848 records)

Model	True Short Stay	False Long Stay	False Short Stay	True Long Stay
Random Forest	129,342	8,134	32,781	124,591
XGBoost	132,078	5,398	34,692	122,680
CatBoost	131,642	5,834	34,709	122,663
MLP	129,918	7,558	32,424	124,948

Looking at where the models actually disagree is more informative than the headline accuracy numbers. XGBoost makes the fewest false-positive errors (5,398), which is why it edges out the others on precision, with CatBoost close behind at 5,834. On the flip side, Random Forest and the MLP are better at actually catching true long stays (124,591 and 124,948 respectively), which gives them the recall advantage. The MLP ends up with the best balance of the four, correctly identifying more long stays than any other model while still keeping false positives reasonably low.

5.4 Feature Importance

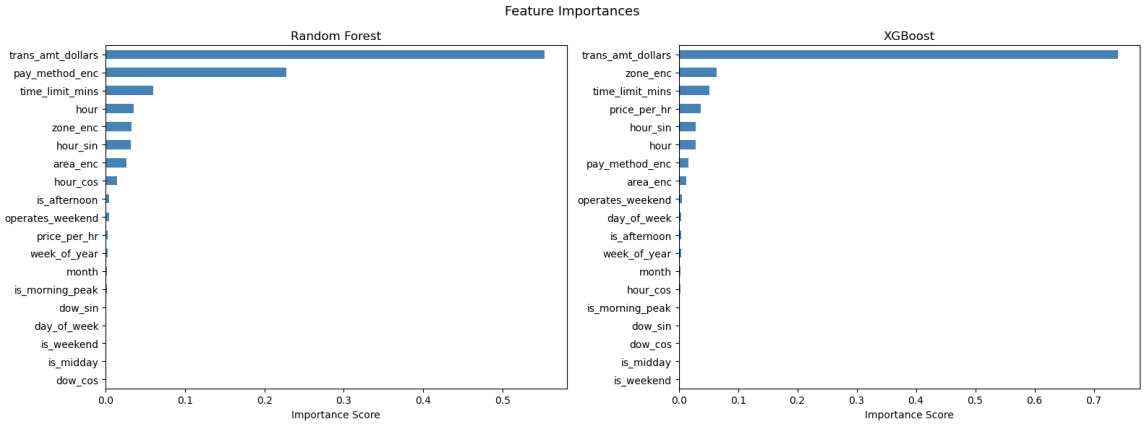

Figure 3: Feature importances for Random Forest, XGBoost, and CatBoost (1×3 grid). The MLP does not produce feature importances and is excluded from this plot.

For Random Forest, the ranking comes out as `trans_amt_dollars` (0.55), `pay_method_enc` (0.22), `time_limit_mins`, `hour`, and `zone_enc`.

XGBoost leans even harder on a single feature: `trans_amt_dollars` (0.73), followed by `zone_enc`, `time_limit_mins`, `price_per_hr`, and `hour_sin`.

It is not hard to see why `trans_amt_dollars` dominates both models. If someone pays for two hours of parking up front, that is a fairly direct signal they are planning to stay a while. At the same time, this raises a fair concern: the amount paid is almost mechanically linked to duration, since more time generally costs more money. That makes it feel like a slightly unfair advantage for the model rather than a genuinely learned pattern, which is what motivated the ablation study described next.

6 Ablation Study: Removing `trans_amt_dollars`

6.1 Motivation

Given that `trans_amt_dollars` accounts for more than half of the feature importance in both tree-based models, it is worth checking what remains once it is taken out of the picture. There is also a practical angle to this: in a real deployment, stay type might need to be predicted before or during a parking session, not after the driver has already paid, in which case the transaction amount would not even be available yet.

To test this, all four models were retrained on the identical dataset with `trans_amt_dollars` dropped from the feature set, then evaluated on the same 294,848-row test split used throughout this report.

6.2 Results

```
============================================================
  SIDE-BY-SIDE: WITH vs WITHOUT trans_amt_dollars
============================================================
       Model  Accuracy (with amount)  F1 Score (with amount)  Accuracy (without amount)  F1 Score (without amount)
Random Forest                 0.8612                  0.8590                     0.8074                     0.8043
      XGBoost                 0.8640                  0.8596                     0.8084                     0.8065
     CatBoost                 0.8625                  0.8582                     0.8067                     0.8039
          MLP                 0.8644                  0.8621                     0.8093                     0.8097
============================================================
```

Figure 4: Model performance with versus without `trans_amt_dollars`: accuracy and F1 score for Random Forest, XGBoost, CatBoost, and MLP.

As Figure 4 shows, every model takes a real hit. Accuracy falls from 0.8612 to 0.8074 for Random Forest, from 0.8640 to 0.8084 for XGBoost, from 0.8625 to 0.8067 for CatBoost, and from 0.8644 to 0.8093 for the MLP. F1 scores move in the same direction: 0.8590 down to 0.8043 for Random Forest, 0.8596 down to 0.8065 for XGBoost, 0.8582 down to 0.8039 for CatBoost, and 0.8621 down to 0.8097 for the MLP.

6.3 Interpretation

Losing 5 to 6 percentage points across the board confirms that `trans_amt_dollars` was carrying a substantial share of the model's performance, roughly in line with how strongly it dominated the feature importance charts. More encouragingly, accuracy still holds around 80% without it. That remaining performance has to be coming from genuinely useful signals, zone, time of day, and meter location, rather than from a single shortcut variable.

One result stands out as unexpected. Without `trans_amt_dollars`, the MLP comes out on top of all four models (F1 of 0.8097), narrowly beating the tree-based models, which land between 0.8039 and 0.8065. A reasonable interpretation is that once the dominant, fairly linear signal from transaction amount is removed, the MLP is better positioned to pick up on the more tangled, non-linear relationships between zone, hour, and the cyclical time features. So the MLP's strong showing earlier was apparently not just a case of riding the same easy signal as the tree models; it appears to be finding something the others are not.

7 Discussion

Stepping back, all four models clear a naive baseline by a comfortable margin, and the fact that they converge on almost the same accuracy suggests the limiting factor here is the information available in the features, not weaknesses in any particular algorithm. When

every model family lands in roughly the same place, that usually points to the data, not the modeling choices, as the real bottleneck.

The single biggest driver of the result is `trans_amt_dollars`. That a driver paying more up front tends to mean a longer stay is not exactly a surprising discovery, but the size of the effect, 55 to 73% of feature importance in the tree models, is larger than might be expected going in. Payment method also plays a real role, ranking second for Random Forest. A plausible reading of this is that credit card users are simply more comfortable making a larger payment for a longer block of time than someone paying in coins.

Time-of-day features matter too, just less dramatically. The dip at 5pm, down to an 18% long-stay rate, is one of the more interesting small details in the data, and likely reflects drivers grabbing a quick spot for a short errand right before evening enforcement ends.

A limitation worth acknowledging: this dataset only captures transactions where someone actually paid. Free parking, which is common in residential streets and on Sundays across San Diego, simply does not appear here at all. Getting a fuller picture of city-wide parking behaviour would mean pairing this kind of transaction data with actual occupancy sensors.

8 Conclusion and Future Work

Overall, this project shows that ordinary parking transaction records, data cities are already collecting, can support a useful prediction task with over 86% accuracy. Training four quite different models on the same problem and seeing them converge gives reasonable confidence that this result is not a fluke tied to one particular algorithm. The ablation study added an important caveat: a large share of that accuracy depends on the transaction amount, a feature that will not always be available in a real deployment setting.

There are a few directions worth pursuing given more time. Wrapping the trained model in a small REST API could make it usable inside a navigation app, giving drivers live guidance rather than a static report. Bringing in external context, local events or weather, might help on days that do not follow the usual pattern. Adding lag features, such as occupancy over the previous 30 minutes, could capture short-term dependencies that the current feature set misses entirely. It would also be worth testing this same approach on data from another city, to see whether these patterns are specific to San Diego or hold up more generally. Finally, a more capable MLP, or a sequence model like an LSTM applied directly to time-ordered transactions, might squeeze out patterns that tree-based models simply cannot reach.

References

[1] Zheng, Y., Rajasegarar, S., & Leckie, C. (2015). *Parking availability prediction for sensor-enabled car parks in smart cities*. In *Proceedings of the IEEE 10th International Conference on Intelligent Sensors, Sensor Networks and Information Processing*. IEEE.

[2] Shoup, D. C. (2006). *Cruising for parking. Transport Policy*, 13(6), 479–486.

[3] Ji, Y., Tang, D., Blythe, P., Guo, W., & Wang, W. (2014). *Short-term forecasting of available parking space using wavelet neural network model. IET Intelligent Transport Systems*, 9(2), 202–209.

[4] Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[5] University of California, Irvine. (2016). *Parking Birmingham Data Set*. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/Parking+Birmingham

[6] Breiman, L. (2001). *Random forests. Machine Learning*, 45(1), 5–32.

[7] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). *CatBoost: Unbiased boosting with categorical features*. In *Advances in Neural Information Processing Systems* (Vol. 31).

[8] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors. Nature*, 323(6088), 533–536.